An Interpretable AI-Based Observability Framework for Predicting User Experience Degradation in Digital Systems

Digital Health Index Forecasting and Automated Operational Response

MARCELO LISBOA ROCHA*

Postgraduate Program in Governance and Digital Transformation, Federal University of Tocantins (UFT), Brazil, mlisboa@uft.edu.br

NISIA TEIXEIRA ANDRADE

Postgraduate Program in Governance and Digital Transformation, Federal University of Tocantins (UFT), Brazil, nisia.andrade@uft.edu.br

RIBERVAL SARAIVA DA SILVA

Postgraduate Program in Governance and Digital Transformation, Federal University of Tocantins (UFT), Brazil, riberval.saraiva@uft.edu.br

SIDNEI R. F. FELICIANO

Postgraduate Program in Governance and Digital Transformation, Local Manager in Magistrates' School of Rondonia (EMERON), Brazil, sfeliciano@tjro.jus.br

This study presents an interpretable AI-based observability framework for anticipating user experience degradation in digital systems. Observability metrics exported from Dynatrace are standardized and consolidated into a Digital Health Index (DHI) on a 0–100 scale. Using 2,158 hourly records collected from December 2025 to February 2026, the framework evaluates next-hour DHI regression and complementary degradation classification under a chronological validation protocol. The tuned Random Forest Regressor achieved MAE = 7.2764, RMSE = 9.2063, and R^2 = 0.4745, while the evaluated LSTM regressor achieved MAE = 33.3319, RMSE = 35.6617, and R^2 = -6.9117. In the classification task, Random Forest obtained ROC-AUC = 0.8649 and F1-score = 0.5859; the LSTM classifier did not identify positive degradation cases at the adopted threshold. Interpretability is supported through DHI weights, observed risk contributions, and feature-group ablation. Finally, prediction artifacts are consumed by an n8n workflow for criticality assessment, alert routing, and historical logging. The results indicate that an interpretable tabular approach, combined with observability and automation, can support proactive digital-service management while preserving operational transparency.

CCS CONCEPTS • Computing methodologies → Machine learning • Information systems → Data analytics • Applied computing → Enterprise computing

ACM Reference Format:
Marcelo Lisboa Rocha, Nisia Teixeira Andrade, Riberval Saraiva da Silva, and Sidnei R. F. Feliciano. 2026. An Interpretable AI-Based Observability Framework for Predicting User Experience Degradation in Digital Systems. In ICOAI 2026. 16 pages.

Additional Keywords and Phrases: Observability; Machine learning; User experience; Digital health index; Forecasting; Automation; AIOps

Corresponding author: Marcelo Lisboa Rocha, mlisboa@uft.edu.br. Postal address: Quadra 604 Sul, Al. 12, No. 29, Palmas/TO, Brazil, 77022-032. Telephone: +55 63 98405-5547.

1 INTRODUCTION

Digital services increasingly mediate institutional work, making user experience a component of service continuity rather than a peripheral quality attribute. Modern observability platforms make large volumes of performance, error, interaction, and infrastructure data available, but these signals are commonly interpreted in separate dashboards. A favorable aggregate indicator can therefore coexist with localized latency, interaction failures, resource saturation, or unstable behavior that affects specific time windows and user groups [1, 2].

Apdex is useful because it translates response-time behavior into an accessible satisfaction score [3]. However, a single synthetic measure cannot represent all dimensions of digital health. The monitored environment in this study illustrates the problem: mean Apdex was 0.9331, yet the mean DHI was 69.3718, with 74 critical hours and 561 hours classified as warning or worse. This contrast motivates an integrated index that preserves the contribution of heterogeneous operational variables instead of treating any one metric as a complete description of user experience.

The article therefore proposes an interpretable AI-based observability framework that performs four connected functions: construction of a Digital Health Index; prediction of next-hour DHI; classification of impending degradation; and operationalization of the results through workflow automation. The central research question is whether heterogeneous observability data can be transformed into an interpretable predictive signal that is sufficiently accurate for proactive monitoring and sufficiently transparent for institutional use.

The contribution is architectural and methodological rather than the proposal of a new learning algorithm. It combines robust data treatment, a weighted composite index, chronological validation, comparison of tabular and recurrent models, and an automation layer that translates predictions into action. This integration addresses the gap between analytical experiments and operational monitoring by connecting detection, interpretation, communication, and auditability in one reproducible flow.

2 BACKGROUND AND RESEARCH GAP

2.1 Observability, user experience, and composite indicators

Observability supports inference about the internal state of a digital system through externally available signals such as metrics, logs, and traces. In user-facing applications, meaningful interpretation requires the joint reading of response time, loading behavior, errors, sessions, interaction patterns, and resource use. Dynatrace consolidates many of these measurements, but the analytical problem remains: values have different units, distributions, directions, and operational meanings. Visibility alone does not guarantee a coherent assessment of system health [4, 2].

Composite indicators provide a defensible mechanism for placing heterogeneous variables on a common scale, provided that normalization, weighting, and interpretation are explicit [5]. The proposed DHI follows this principle. It retains Apdex as a user-perception signal but complements it with latency, interaction, error, session, and infrastructure metrics. The resulting score is designed to be inspectable: each metric has a declared methodological weight, and its observed contribution to risk can be calculated for the analyzed dataset.

2.2 Interpretable prediction and operational response

Random Forest is appropriate for this setting because it captures nonlinear relationships and interactions without eliminating the possibility of feature-level interpretation [6]. LSTM networks provide a contrasting

sequential approach that can learn temporal dependence directly from ordered windows [7]. Recent applied forecasting research also shows how explainable machine learning can link predictive outputs to interpretable driver analyses in decision contexts [8]. Comparing the two does not assume that one family is universally superior; it tests which representation is more suitable for the available dataset, monitoring horizon, and validation protocol. Recent work on multivariate time-series anomaly detection similarly emphasizes temporal context, adaptive models, and explainability in complex systems [9, 10, 11, 12].

A second gap concerns the distance between model output and institutional action. Detection systems become more useful when alerts are correlated, filtered, routed, and recorded rather than merely displayed [13, 14]. In the proposed framework, n8n consumes analytical artifacts, assigns criticality, sends notifications, and maintains an execution history. The novelty lies in combining user-experience observability, an interpretable composite index, predictive modeling, and automated response within one applied architecture.

2.3 Research positioning and design requirements

The literature establishes useful components for the present problem but does not, by itself, resolve their integration. Observability platforms provide detailed signals; composite-indicator research provides rules for aggregation; machine learning provides prospective inference; and workflow systems provide automation. When these components are developed independently, however, the result may remain analytically fragmented: a dashboard without prediction, a model without operational meaning, or an alert without an explanation of the conditions that produced it. The research gap is therefore located at the junction of these capabilities.

The framework was designed around four requirements derived from that gap. First, heterogeneous metrics must be transformed into comparable risk components without hiding their origin. Second, validation must preserve temporal order so that reported performance reflects a prospective monitoring scenario. Third, the model must support interpretation at both the feature-group and individual-metric levels. Fourth, the analytical result must be expressed in a form that an automated workflow can consume, classify, communicate, and audit. These requirements make interpretability an architectural property rather than an optional visualization.

Accordingly, the study does not treat DHI, Random Forest, LSTM, or n8n as isolated contributions. DHI supplies the common operational target; the tabular and recurrent models test alternative representations of time; ablation and contribution analysis explain the predictive and methodological roles of the variables; and n8n closes the loop between inference and response. The evaluation asks whether this arrangement is coherent and useful under the available data, while explicitly avoiding claims of universal superiority or causality.

3 METHODOLOGY

3.1 Framework and dataset

This applied study designed and evaluated a two-layer observability framework. The analytical layer was implemented in Python/Google Colab and performs parsing, normalization, DHI calculation, feature engineering, model training, evaluation, and artifact export. The operational layer was implemented in n8n and performs scheduled artifact retrieval, criticality interpretation, multichannel notification, and historical logging. Figure 1 summarizes the flow from Dynatrace data to automated incident response.

The empirical dataset contains 2,158 hourly observations from December 1, 2025 through February 28, 2026. Variables include Apdex, page load time, load-action response time and duration, XHR action duration

and response end, first input delay, session count and duration, CPU usage, total errors, errors during user actions, JavaScript errors, rage clicks, and error clicks. The data represent a real institutional digital environment and were analyzed only after standardized naming, unit conversion, and consistency checks.

The hourly grain was retained because it matches the intended near-term decision horizon and avoids mixing observations recorded at incompatible intervals. Duplicate timestamps and naming inconsistencies were checked before feature construction. The supervised target was created by shifting DHI by one hour. Consequently, each predictor row describes the present and recent past, while its label describes the subsequent hour. This alignment is essential for preventing information from the forecast horizon from entering the predictors.

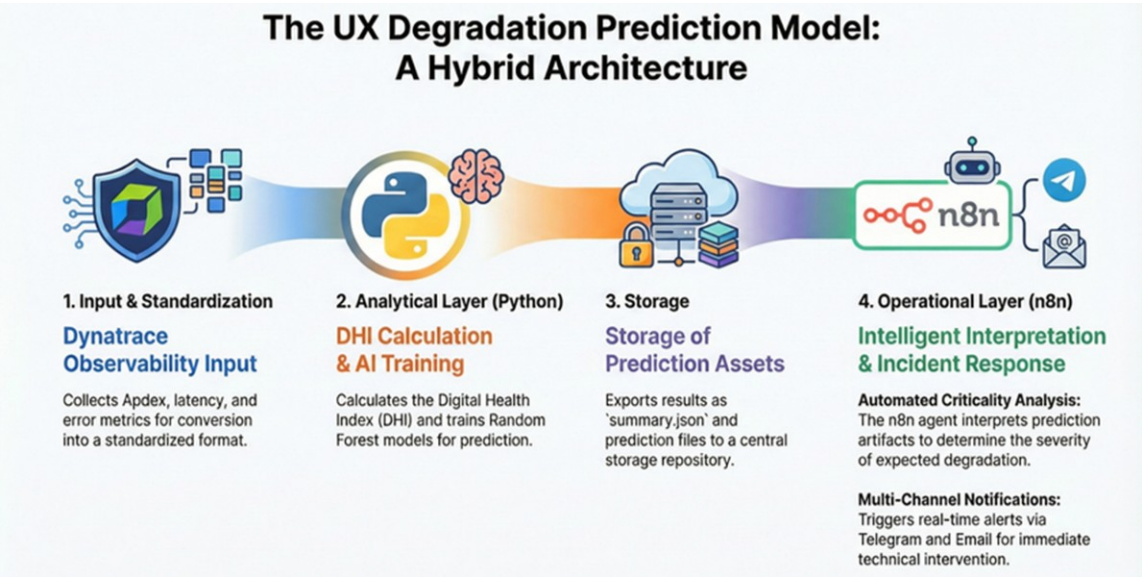


Figure 1: Proposed hybrid architecture integrating the Python analytical layer and the n8n operational layer.

3.2 Data treatment and DHI construction

Time values expressed in seconds, milliseconds, microseconds, and minutes were converted to a common unit; abbreviated counts were converted to absolute values; missing values were imputed with the variable median; and month, day, hour, and weekday were derived. Robust normalization used the 10th and 90th percentiles to limit the influence of extreme observations. For metrics in which higher values indicate degradation, the normalized risk component was calculated as follows:

$$r_{i,t} = \min(1, \max(0, (x_{i,t} - P10_i)/(P90_i - P10_i))) \quad (1)$$

For Apdex, whose direction is reversed, risk was calculated by subtracting the normalized value from one:

$$r_{i,t} = 1 - \min(1, \max(0, (x_{i,t} - P10_i)/(P90_i - P10_i))) \quad (2)$$

The global risk score and DHI were then defined using declared weights w_i:

$$Risk_t = 100 \times \Sigma_i \, w_i r_{i,t} \quad (3)$$

$$DHI_t = 100 - Risk_t \quad (4)$$

DHI was interpreted in four operational ranges: healthy (DHI ≥ 75), stable (60 ≤ DHI < 75), warning (45 ≤ DHI < 60), and critical (DHI < 45). The weighting scheme balances user perception, performance, errors, interactions, and infrastructure. It also provides an interpretable basis for explaining why the index changes, because the contribution of every normalized metric remains recoverable.

3.3 Prediction tasks and validation protocol

The main task predicts next-hour DHI. Observations were chronologically ordered and enriched with DHI lags, moving averages, moving standard deviations, and calendar attributes. This feature-engineering emphasis is consistent with applied studies that combine numerical attributes, feature extraction, and multiple machine-learning algorithms to improve predictive performance [15]. The first 80% of observations were used for training and the final 20% for testing, with no random shuffling. Nine tabular regressors were compared, and Random Forest was tuned with RandomizedSearchCV and TimeSeriesSplit so that every validation fold preserved temporal order [16]. The random seed was 42.

The recurrent comparison used 24-hour windows normalized with MinMaxScaler and an LSTM architecture with early stopping. Regression performance was assessed with MAE, RMSE, and R^2. A complementary classification task defined a positive case as next-hour DHI below 60 and compared Random Forest and LSTM using accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices. The analysis was executed with Python 3.12, scikit-learn 1.6.1, TensorFlow/Keras 2.20.0, pandas 2.2.2, NumPy 2.0.2, and matplotlib 3.10.0; PyCaret was used only as complementary automated validation [17, 18].

RandomizedSearchCV examined n_estimators = [100, 200, 300], max_depth = [None, 8, 12, 16], min_samples_split = [2, 5, 10], min_samples_leaf = [1, 2, 4], and max_features = [sqrt, log2, None]. The selected configuration used 300 trees, min_samples_split = 5, min_samples_leaf = 1, max_features = sqrt, and unrestricted depth. Keeping the final chronological test segment outside tuning reduced the risk that hyperparameter selection would exploit future observations.

MAE expresses average absolute error in DHI points, RMSE emphasizes larger deviations, and R^2 measures the represented variance relative to a mean baseline. For classification, precision and recall were considered together because monitoring must balance unnecessary alerts against missed degradation. ROC-AUC summarizes ranking quality independently of one threshold, while the confusion matrix exposes the counts that determine the practical consequence of each estimator.

3.4 Interpretability, ablation, and automation

Interpretability was evaluated at two levels. Methodological importance corresponds to the weights specified during DHI construction, whereas observed contribution corresponds to the mean realized share of each metric in the risk score. Methods such as SHAP, ICE, PDP, and ALE complement this type of explanation by making global and local prediction behavior inspectable [19]. An ablation analysis removed DHI lags, moving statistics, Apdex, error metrics, and interaction metrics one group at a time while retaining the same split, seed, tuned estimator, and metrics. This design distinguishes structural relevance from empirical predictive dependence.

After inference, Python exports summary.json, model-comparison files, contribution tables, and prediction files. A scheduled n8n workflow retrieves the most recent artifacts, maps results to no-alert, moderate-alert, or critical-alert categories, routes messages through Telegram and email, and records execution metadata. The operational layer therefore consumes the analytical result without altering model training and provides an auditable separation between inference and response [20].

The decision logic intentionally remains external to the learning algorithm. A model produces predicted DHI or degradation probability; the workflow applies institutional thresholds and communication rules. This separation permits alert criteria to be revised without retraining the estimator and allows the same analytical artifact to support different destinations. Stored records include the processed file, timestamp, assigned criticality, and routed message, providing a basis for later audit and threshold calibration.

4 RESULTS AND DISCUSSION

4.1 Digital-health profile of the monitored period

The dataset presented a mean Apdex of 0.9331 and a mean DHI of 69.3718. DHI ranged from 33.7865 to 99.2780; 74 hours were critical, 561 were in warning or worse, and 815 were healthy (Table 1). The globally favorable Apdex therefore did not eliminate substantial intervals of reduced digital health. DHI exposed these periods because it incorporated performance, error, interaction, and infrastructure evidence that was not contained in Apdex alone.

Table 1: Summary of the dataset's descriptive results.

Indicator	Value
Records analyzed	2,158
Initial period	2025-12-01 02:00:00
Final period	2026-02-28 23:00:00
Mean Apdex	0.9331
Mean DHI	69.3718
Minimum DHI	33.7865
Maximum DHI	99.2780
Critical hours	74
Hours in warning or worse	561
Healthy hours	815

The Apdex series in Figure 2 remained close to its favorable baseline during most of the period, but recurrent sharp decreases reveal short-lived disruptions that are obscured by the overall mean. The temporal pattern is consistent with a service that is generally satisfactory but not uniformly healthy. For monitoring purposes, these local declines matter because users experience the system at particular times rather than as a three-month average.

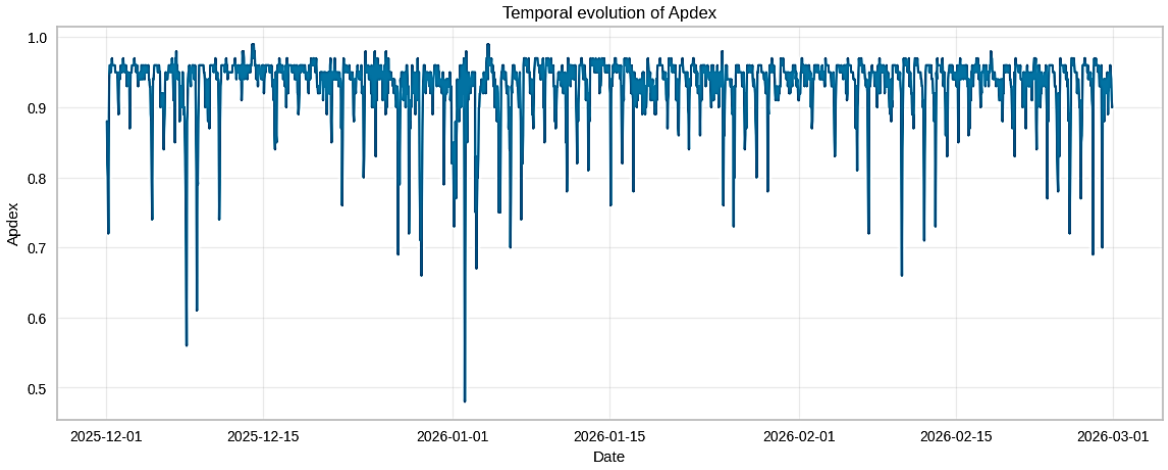


Figure 2: Temporal evolution of Apdex during the analyzed monitoring period.

The distribution of DHI states in Figure 3 reinforces this interpretation. Healthy and stable observations predominated, but the warning and critical groups were large enough to justify anticipatory monitoring. The monthly mean also varied, with January below December and February. These findings support treating digital health as a temporal, multivariate phenomenon rather than a fixed property of the service.

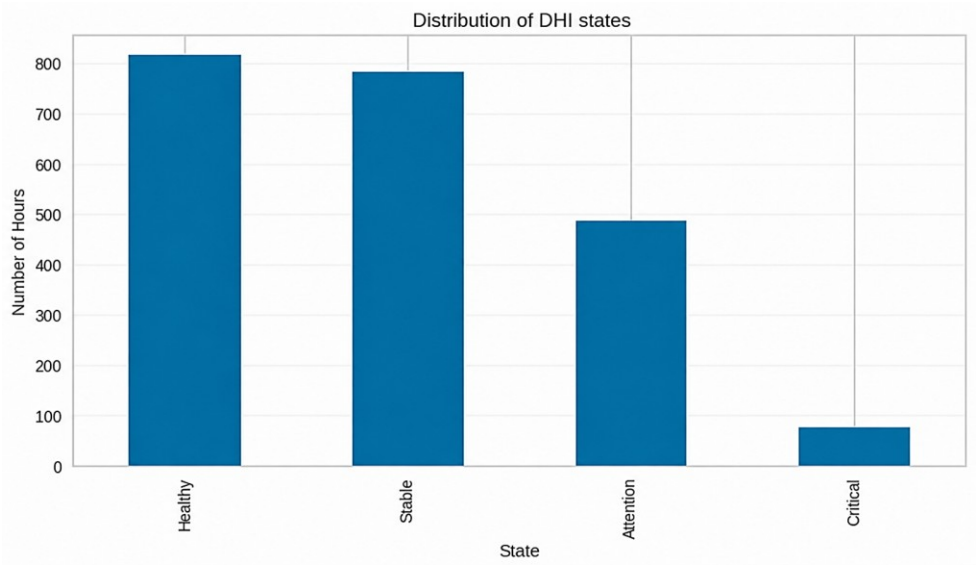


Figure 3: Distribution of DHI states during the analyzed monitoring period.

4.2 Correlations and exploratory evidence

Exploratory correlations, as show in Figure 4, were used to characterize association, not to establish causality or select features in isolation. Apdex had the strongest positive correlation with DHI (r = 0.5174). Page Load

Time (r = -0.5179) and Response Time - load action (r = -0.5169) showed the strongest negative relationships, followed by Load Action Duration (r = -0.3910), Rage Click Count (r = -0.3609), Error Clicks (r = -0.3561), Session Count (r = -0.3482), and Error Count (r = -0.3452). The directions are operationally coherent: longer response and loading times, errors, and signs of user frustration coincide with lower digital health.

The matrix also shows substantial correlation among error-related and interaction-related variables. This dependence helps explain why individual coefficients should not be interpreted as independent causal effects. It also supports the use of a multivariate estimator capable of modeling nonlinear relationships and feature interactions. In this framework, correlation is a descriptive bridge between raw observability data and the subsequent analyses of predictive dependence, ablation, and risk contribution.

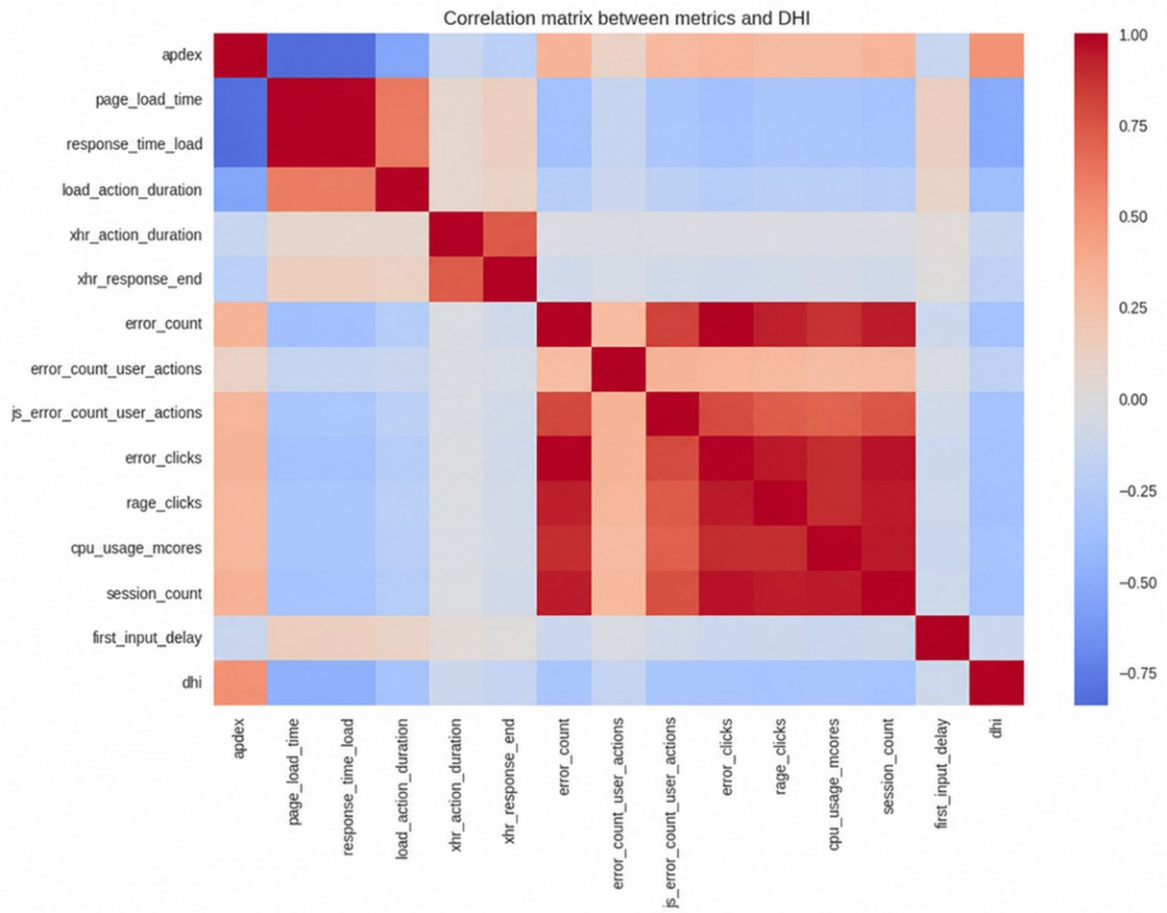


Figure 4: Correlation matrix between observability metrics and DHI.

4.3 Next-hour DHI regression

Extra Trees and Random Forest produced the strongest initial tabular results. After temporal tuning, the Random Forest Regressor achieved MAE = 7.2764, RMSE = 9.2063, and R^2 = 0.4745 (Table 2). Its R^2 indicates

moderate explanatory capacity rather than a complete model of system behavior, but the error levels and the actual-versus-predicted series in Figure 5 show that it followed a meaningful portion of short-term DHI variation. The model should consequently be interpreted as decision support for prospective monitoring, not as a deterministic forecast.

Table 2: Regression performance of representative models.

Model	MAE	RMSE	R^2
Extra Trees	7.2705	9.2412	0.4706
Random Forest (initial)	7.2154	9.2579	0.4686
Linear Regression	7.4161	9.4213	0.4497
Random Forest (tuned)	7.2764	9.2063	0.4745
LSTM	33.3319	35.6617	-6.9117

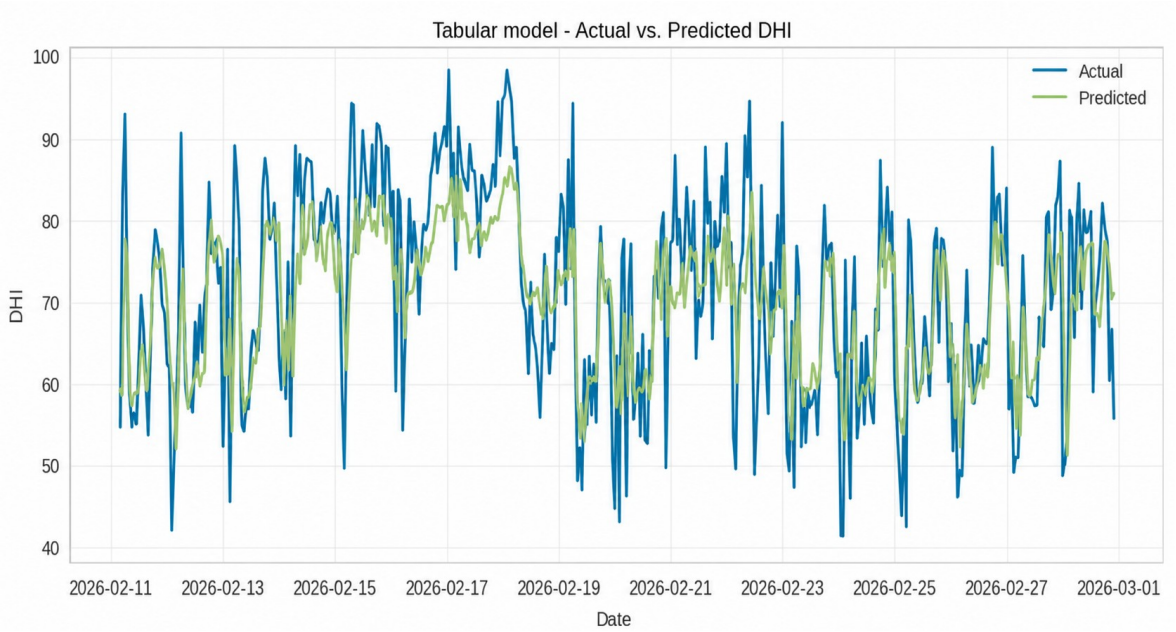


Figure 5: Actual versus predicted DHI values from the tuned Random Forest Regressor.

The predicted series is smoother than the observed series, particularly around abrupt peaks and troughs. This behavior is expected from an ensemble optimized for average out-of-sample performance: it captures central movement more consistently than rare extremes. The residual plot in Figure 6 shows errors distributed on both sides of zero across the prediction range, but with several large deviations. Those deviations are operationally important because sudden degradation may be precisely the event that requires an alert.

The regression results should therefore be read as evidence of useful but incomplete predictability. A model with moderate R^2 can still support prioritization when its errors are understood and its outputs are combined with explicit thresholds and human review. Conversely, the framework should not suppress direct observability signals merely because a forecast indicates stability. Prediction complements, rather than replaces, current-state monitoring.

The LSTM regressor achieved MAE = 33.3319, RMSE = 35.6617, and R^2 = -6.9117 under the adopted configuration. This result does not establish a general limitation of recurrent networks; it shows that, for this dataset and window, the tabular representation of recent history through lags and moving statistics was more effective than the evaluated sequence model. The comparison highlights the importance of matching model complexity to sample size, temporal horizon, and feature representation.

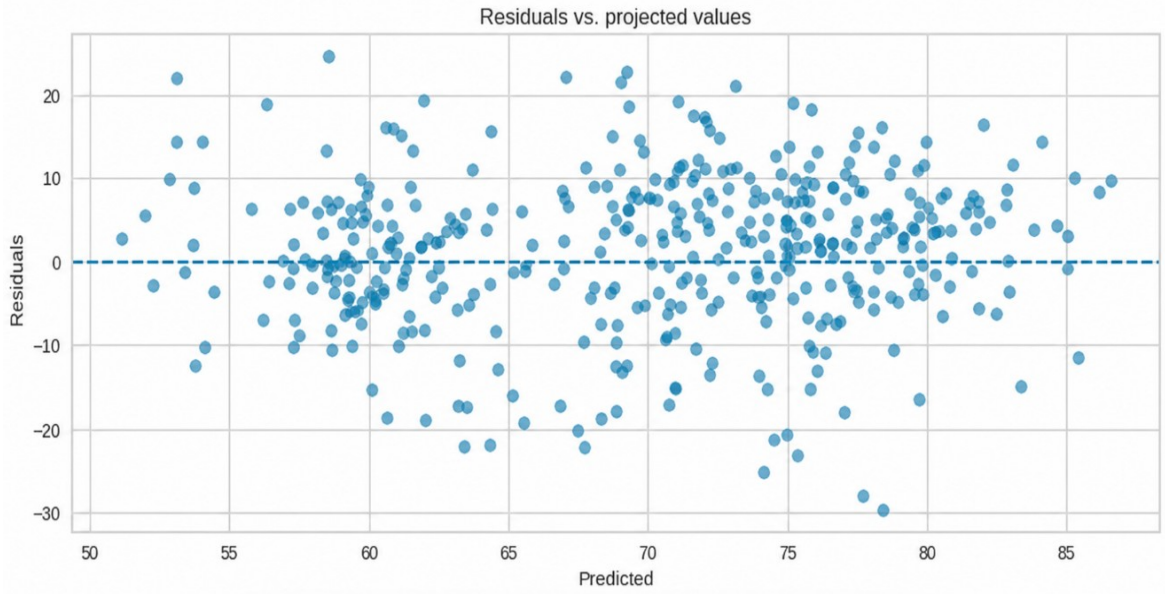


Figure 6: Residuals versus predicted DHI values for the tuned Random Forest Regressor.

4.4 Ablation and metric importance

Ablation confirmed that temporal context was central to next-hour prediction (Table 3). Removing DHI lags increased MAE from 7.2764 to 8.9521 and reduced R^2 from 0.4745 to 0.2244. Removing moving statistics reduced R^2 to 0.3342, while excluding Apdex reduced it to 0.3805. Error and interaction groups produced smaller but consistent losses. Predictive capacity therefore depended on the combination of recent index history, short-term trends, user perception, errors, and interactions rather than a single signal.

Table 3: Feature-group ablation for next-hour DHI forecasting.

Configuration	MAE	RMSE	R^2
Full feature set	7.2764	9.2063	0.4745
Without DHI lags	8.9521	11.1842	0.2244
Without moving statistics	8.3876	10.3627	0.3342
Without Apdex	7.9963	9.9974	0.3805
Without error metrics	7.7748	9.7818	0.4028
Without interaction metrics	7.5719	9.5929	0.4301

The DHI weighting scheme assigned the greatest methodological importance to Apdex (18%) and Page Load Time (12%). Observed contribution followed the same broad pattern, with Apdex accounting for 17.8494% of average risk and Page Load Time for 11.9319%. XHR action duration contributed more empirically (9.2310%) than its declared weight (6%), showing why structural weight and realized contribution should be reported separately. Table 4 summarizes the eight most influential metrics across both views.

Table 4: Methodological importance and mean observed contribution to risk.

Metric	Methodological weight (%)	Observed share (%)
Apdex	18.0	17.8494
Page Load Time	12.0	11.9319
Load Action Duration	8.0	8.5314
Response Time - load action	8.0	7.8489
Error Count	8.0	7.1705
Error Count during user actions	7.0	7.0143
Error Clicks	7.0	5.8372
Action Duration - XHR	6.0	9.2310

Figure 7 provides the full methodological ranking. The distribution is deliberately not uniform: higher weights are assigned to user-perception and loading variables, while error, interaction, and infrastructure variables supply complementary evidence. Reporting the ranking makes the normative assumptions of DHI visible and gives domain experts a concrete object for validation or recalibration in another institutional environment.

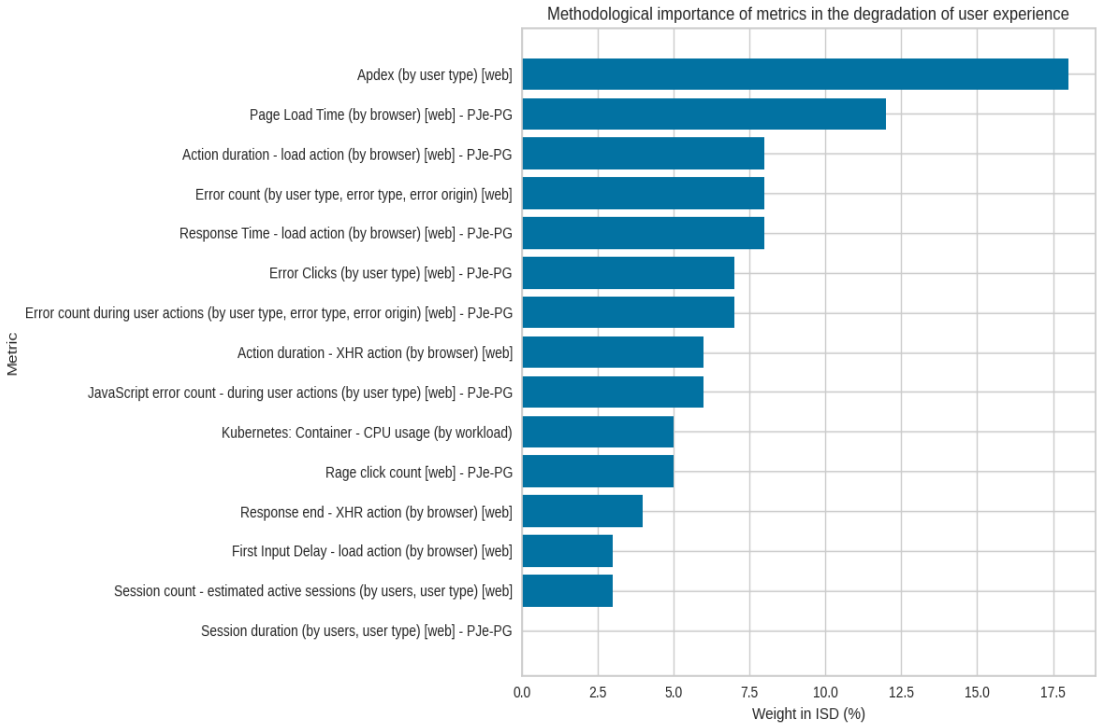


Figure 7: Methodological importance of the metrics used to construct DHI.

4.5 Degradation classification

For the classification task, Random Forest achieved accuracy = 0.8080, precision = 0.5859, recall = 0.5859, F1-score = 0.5859, and ROC-AUC = 0.8649 (Table 5). The LSTM classifier achieved accuracy = 0.7701 but zero precision, recall, and F1-score at the adopted threshold, because it did not predict positive degradation cases. The contrast demonstrates why accuracy alone is inadequate for imbalanced operational data.

Table 5: Comparison of degradation-classification performance.

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Random Forest	0.8080	0.5859	0.5859	0.5859	0.8649
LSTM	0.7701	0.0000	0.0000	0.0000	0.4975

The confusion matrices in Figure 8 make the operational trade-off explicit. Random Forest correctly identified 287 non-degraded and 58 degraded cases, with 41 false positives and 41 false negatives. The evaluated LSTM labeled every observation as non-degraded, producing 97 false negatives. Because false negatives correspond to degradation episodes that would not trigger a response, the Random Forest result is materially more useful for proactive monitoring even though its recall remains an area for improvement.

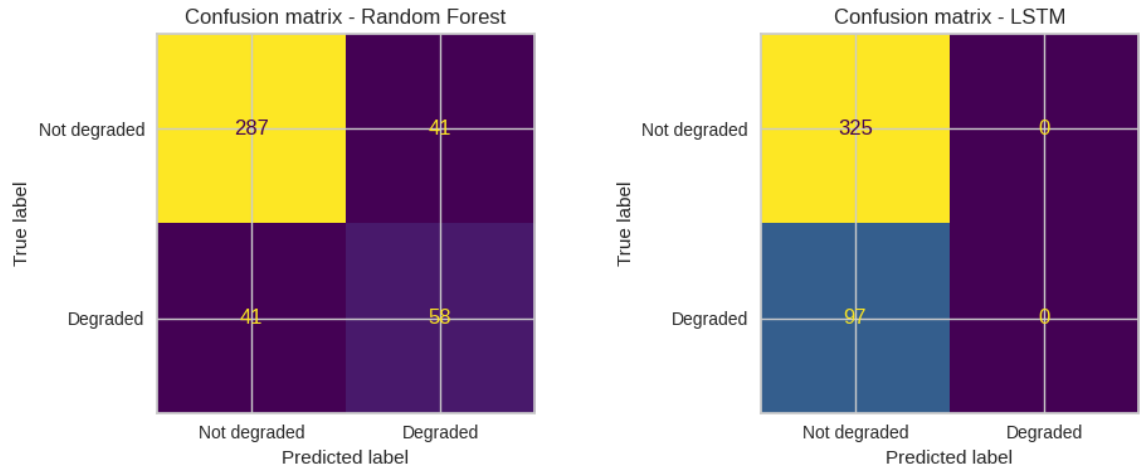


Figure 8: Confusion matrices of the Random Forest and LSTM classifiers.

4.6 Operationalization and institutional implications

The results support an operational sequence in which DHI provides a common language for digital health, Random Forest estimates near-term risk, and n8n converts analytical artifacts into an institutional response. The automation layer can distinguish routine conditions from warning and critical scenarios, send rapid Telegram alerts, create a formal email record, and preserve execution history. Interpretability is retained because alerts can be accompanied by current and predicted DHI, criticality, and the metrics contributing most to risk.

The separation between analytical and operational layers is central to governance. Python remains responsible for reproducible computation, while n8n applies organizational rules to validated outputs. This design creates clear points of control: model versions and data transformations can be reviewed in the analytical layer; alert thresholds, recipients, and delivery channels can be reviewed in the workflow; and execution records can be used to reconstruct why a notification was issued. The arrangement is therefore compatible with incremental deployment and institutional accountability.

In practical use, no-alert conditions can remain in the historical record, moderate alerts can prompt preventive inspection, and critical alerts can trigger immediate communication. Telegram provides a rapid channel, whereas email supports formal documentation. Because the message can include DHI, forecast values, criticality, and dominant risk contributors, the recipient receives both a decision signal and an explanation. This reduces the distance between a statistical output and an actionable monitoring routine.

4.7 Limitations and future work

The findings are exploratory and bounded by a single institutional dataset, a three-month historical window, and spreadsheet-based export. DHI weights require validation in other environments, and recurrent models may benefit from larger samples, alternative windows, imbalance treatment, and architecture tuning. Future work should evaluate direct Dynatrace API ingestion, longer monitoring periods, cross-system validation, calibration

of alert thresholds, and comparison with streaming anomaly-detection methods. Raw operational data cannot be publicly released, but an anonymized schema, code, and synthetic samples can support reproducibility.

The DHI is also partly normative because its weights encode assumptions about the relative importance of each metric. Observed contributions and ablation provide empirical checks, but they do not replace validation with service owners or users. Likewise, the classification threshold determines the balance between false positives and false negatives. Future evaluations should compare thresholds under explicit operational costs, measure alert fatigue, and assess whether predicted degradation corresponds to incident records and reported user experience.

Further work should test alternative sequence lengths, imbalance-handling strategies, calibrated probabilities, and additional temporal models under the same chronological protocol. Direct API ingestion would permit true streaming evaluation and reduce dependence on spreadsheet export. Cross-system studies are required to determine whether the present metric weights and model configuration transfer or should be adapted to different workloads, user populations, and service objectives.

5 CONCLUSION

This study demonstrated the feasibility of an interpretable AI-based observability framework for predicting user experience degradation. Its main point is that digital health cannot be represented adequately by one aggregate metric or by an isolated predictive model. The proposed solution combines heterogeneous signals in a transparent DHI, predicts next-hour behavior under chronological validation, identifies the variables that contribute to risk, and operationalizes the result through an automated workflow.

The empirical evidence supported the tabular strategy. Random Forest outperformed the evaluated LSTM in both regression and classification, while ablation showed that DHI lags and moving statistics were especially important. Apdex remained the most influential metric, but the discrepancy between high mean Apdex and recurrent warning or critical DHI states confirmed the need for multivariate interpretation. The framework's value therefore lies in connecting predictive performance with explanation and operational action.

The results do not justify general claims about model families and should be interpreted within the evaluated data and configuration. Nevertheless, they provide a concrete basis for proactive digital-service management: an institution can detect emerging degradation, understand its principal drivers, route alerts, and preserve an auditable response history. Broader validation will determine how well the framework transfers across systems, longer time horizons, and real-time data streams.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used ChatGPT to support language revision and manuscript organization. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

CRediT authorship contribution statement

N.T.A: Conceptualization, Methodology, Software, Formal analysis, Investigation, Visualization, Writing – original draft; R.S.S: Conceptualization, Methodology, Formal analysis, Writing – original draft; S.R.F.F.: Supervision, Validation, Resources, Writing – review & editing; M.L.R: Supervision, Methodology, Validation, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability statement

The raw observability data cannot be publicly shared because they contain operational information from an institutional digital environment. An anonymized data schema, analytical code, and synthetic sample data may be provided upon reasonable request, subject to institutional constraints.

REFERENCES

[1] Dynatrace. (2023). Apdex ratings. Dynatrace Docs. Retrieved March 16, 2026, from https://docs.dynatrace.com/docs/observe/digital-experience/rum-concepts/scores-and-ratings/apdex-ratings

[2] Sridharan, C. (2018). Distributed systems observability: A guide to building robust systems. O'Reilly Media.

[3] Apdex Alliance. (2007). Apdex technical specification: Version 1.1. Apdex Alliance. https://www.apdex.org/wp-content/uploads/2020/09/ApdexTechnicalSpecificationV11_000.pdf

[4] Dynatrace. (2026). User actions. Dynatrace Docs. Retrieved March 16, 2026, from https://docs.dynatrace.com/docs/observe/digital-experience/rum-concepts/user-actions

[5] Nardo, M., Saisana, M., Saltelli, A., Tarantola, S., Hoffmann, A., & Giovannini, E. (2008). Handbook on constructing composite indicators: Methodology and user guide. OECD Publishing. https://doi.org/10.1787/9789264043466-en

[6] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

[7] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

[8] Chaudhary, M.; Alam, M. A.; Zafar, S.; Abubeker, K. M. (2026). Harnessing Explainable AI and Machine Learning for Dual Predictive Modeling of Car Goodwill and Crop MSP in Price and Policy Forecasting. International Journal of Computer Theory and Engineering, 18(1), 48–67. https://doi.org/10.7763/ijcte.2026.v18.1388

[9] Galli, A., La Gatta, V., Moscato, V., Postiglione, M., & Sperlì, G. (2024). Explainability in AI-based behavioral malware detection systems. Computers & Security, 141, Article 103842. https://doi.org/10.1016/j.cose.2024.103842

[10] Tu, F.-F., Liu, D.-J., Yan, Z.-W., Jin, X.-B., & Geng, G.-G. (2024). STFT-TCAN: A TCN-attention based multivariate time series anomaly detection architecture with time-frequency analysis for cyber-industrial systems. Computers & Security, 144, Article 103961. https://doi.org/10.1016/j.cose.2024.103961

[11] Yang, X., Howley, E., & Schukat, M. (2024). ADT: Time series anomaly detection for cyber-physical systems via deep reinforcement learning. Computers & Security, 141, Article 103825. https://doi.org/10.1016/j.cose.2024.103825

[12] Yu, R., Wang, Y., & Wang, W. (2024). AMAD: Active learning-based multivariate time series anomaly detection for large-scale IT systems. Computers & Security, 137, Article 103603. https://doi.org/10.1016/j.cose.2023.103603

[13] Henriques, J., Caldeira, F., Cruz, T., & Simões, P. (2022). An automated closed-loop framework to enforce security policies from anomaly detection. Computers & Security, 123, Article 102949. https://doi.org/10.1016/j.cose.2022.102949

[14] Tjhai, G. C., Furnell, S. M., Papadaki, M., & Clarke, N. L. (2010). A preliminary two-stage alarm correlation and filtering system using SOM neural network and K-means algorithm. Computers & Security, 29(6), 712–723. https://doi.org/10.1016/j.cose.2010.02.001

[15] Sanaz, S.; Jafari, J.; Emdadi, Y. (2025). Enhancing Tag Recommendation Precision on Stack Overflow Data Warehouse: An Integrated Approach Combining Numeric Attributes, Feature Extraction Techniques, and Multiple Machine Learning Algorithms. International Journal of Computer Theory and Engineering, 17(1), 28–35. https://doi.org/10.7763/ijcte.2025.v17.1366

[16] Scikit-learn Developers. (n.d.). TimeSeriesSplit. scikit-learn. Retrieved March 30, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html

[17] Ali, M. (2020, April). PyCaret: An open-source, low-code machine learning library in Python (Version 3.4.0). PyCaret. https://pycaret.org/

[18] Google. (n.d.). Google Colaboratory FAQ. Retrieved March 17, 2026, from https://research.google.com/colaboratory/faq.html

[19] Sajindra, H.; Abekoon, T.; Buthpitiya, S.; Alahakoon, Y.; Rathnayake, N.; Kantamaneni, K.; Rathnayake, U. (2026). Interpretable Machine Learning for Ambient Temperature Prediction: Insights from SHAP, ICE, PDP, and ALE. International Journal of Computer Theory and Engineering, 18(2), 118–132. https://doi.org/10.7763/ijcte.2026.v18.1394

[20] n8n. (n.d.). Workflows. n8n Docs. Retrieved March 30, 2026, from https://docs.n8n.io/workflows/